



Maximum independent set formation on a finite grid by myopic robots ☆,☆☆

Raja Das ^{ID}*, Avisek Sharma, Buddhadeb Sau

Department of Mathematics, Jadavpur University, Kolkata, India

ARTICLE INFO

Keywords:

Myopic robot
Maximum independent set
Finite grid
Autonomous robots
Robot with lights
Distributed algorithms

ABSTRACT

This work deals with the Maximum Independent Set ($MA\mathcal{X}IS$) formation problem in a finite rectangular grid by autonomous robots. Suppose we are given a set of identical robots, where each robot is placed on a node of a finite rectangular grid $\mathcal{G}$ such that no two robots are on the same node. The $MA\mathcal{X}IS$ formation problem asks to design an algorithm and each robot will move autonomously after executing the algorithm and terminate at a node such that after a finite time the set of nodes occupied by the robots is a maximum independent set of $\mathcal{G}$. We assume that robots are anonymous and they execute the same distributed algorithm.

Previous works solved this problem using one or several door nodes through which the robots enter the grid or the graph one by one and occupy the required nodes. In this work, we propose a deterministic algorithm that solves the $MA\mathcal{X}IS$ formation problem in a more generalized scenario, i.e., when the total number of required robots to form a $MA\mathcal{X}IS$ are arbitrarily placed on the grid. The proposed algorithm works under a semi-synchronous scheduler using robots with only two hop visibility range and only three colors.

1. Introduction

Consider a rectangular area R as a bounded region in the two-dimensional Euclidean plane. We embed a rectangular grid graph $\mathcal{G}$ in that rectangular area R . Let a robot with sensing capability stay on the nodes of $\mathcal{G}$. If the sensing radius of the robot is a , we embed the grid with edge length a so that a robot can sense its immediate upward, immediate downward, immediate left, and immediate right neighbor node along with its position completely. A robot can move to its immediate upward, immediate downward, immediate left, and immediate right neighbor nodes through the edges of $\mathcal{G}$. Now we want to place a set of robots on some nodes of $\mathcal{G}$ such that each node of $\mathcal{G}$ is sensed by at least one robot. Suppose we have such a placement of robots. Now *cost* and *resilience* are the major parameters to consider. We consider the cost as the total number of robots required. Now, if one removes some robots from the placement then some nodes become unsafe, that means, there are some nodes which are not sensing covered by any robot anymore (See Fig. 1).

☆ This article belongs to Section A: Algorithms, automata, complexity and games, Edited by Paul Spirakis.

☆☆ A preliminary version of the paper is published in [1].

* Corresponding author.

E-mail addresses: rajad.math.rs@jadavpuruniversity.in (R. Das), aviseks.math.rs@jadavpuruniversity.in (A. Sharma), buddhadeb.sau@jadavpuruniversity.in (B. Sau).

<https://doi.org/10.1016/j.tcs.2025.115077>

Received 22 July 2024; Received in revised form 22 November 2024; Accepted 12 January 2025

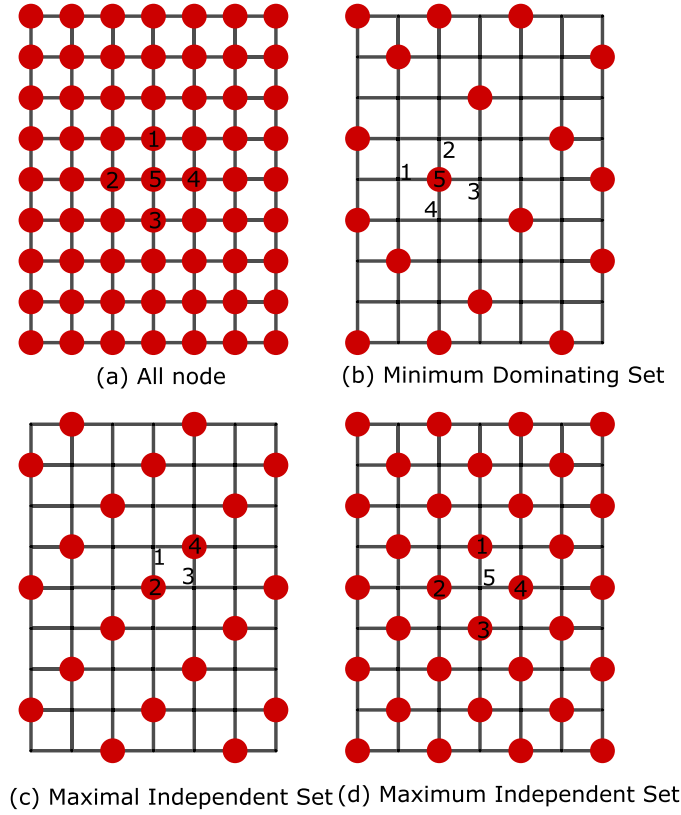


Fig. 1. Various methods of covering (red nodes are robot occupied nodes) [(a): removal of five robots located at node number 1, 2, 3, 4 and 5 will make node number 5 sensing uncovered; (b): removal of one robot located at node number 5 will make node numbers 1, 2, 3, 4 and 5 sensing uncovered; (c): removal of two robots located at node number 2 and 4 will make node numbers 1, 2, 3 and 4 sensing uncovered; (d): removal of four robots located at node number 1, 2, 3 and 4 will make node numbers 1, 2, 3, 4 and 5 sensing uncovered]. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

We consider the resilience (denoted as ρ) as the minimum of the ratio

$$\frac{\text{number of robots removed}}{\text{number of nodes become unsafe}}.$$

We can accomplish the target in different ways. One way can be by putting robots at each node. In this way, the resilience is highest ($\rho = 1$) but the cost is maximum (See Fig. 1(a)). If we put robots on a minimum dominating set of $\mathcal{G}$ then the cost is minimum but resilience is the lowest ($\rho = 1/5$) (See Fig. 1(b)). If we put robots on a maximal independent set of $\mathcal{G}$ then we get a decent resilience ($\rho = 1/3$) (See Fig. 1(c)). If we put robots on a maximum independent set of $\mathcal{G}$, the rectangular area R will be fully sensing covered. The cost in this case is half of the number of nodes and this method gives good resilience ($\rho = 1/2$) (See Fig. 1(d)). So in this work, we consider robots placing on a maximum independent set of $\mathcal{G}$.

In this paper, we give an algorithm for Maximum Independent Set (*MAXIS*) formation on a finite grid. Let a swarm of autonomous robots be placed initially on the distinct nodes of $\mathcal{G}$. The *MAXIS* formation problem asks the robots to rearrange and take positions such that the robot occupied nodes form a *MAXIS* of $\mathcal{G}$. The robots work autonomously, which means they work without any central control. The robots are homogeneous (i.e., they all run the same algorithm), identical (indistinguishable), and anonymous (without any identifier). Such robot swarms are capable of doing certain tasks like gathering, dispersion, exploration, pattern formation, filling, etc. In some cases, robots have memory and can communicate with other robots. Based on these powers there are four types of robot models which are *OBLLOT*, *FSTA*, *FCOM*, *LUMI*. In *OBLLOT* model robots are silent (no communication) and oblivious (no persistent memory). In *FSTA* model robots are silent and non-oblivious. In *FCOM* model robots can communicate but are oblivious. In *LUMI* model robots can communicate and are non-oblivious. Robots can have a finite bit of memory which is generally interpreted as a finite number of lights that can take finitely many different colors. Seeing own light is equivalent to having memory and seeing the lights of other robots is equivalent to communication. After activation, each robot follows a look-compute-move (LCM) cycle. In the look phase, the robot takes a snapshot of its surrounding in its vicinity and gets the position and states of other robots. In compute phase it runs the algorithm and gets an output. In the move phase, the robot moves to its destination node or stays at the same node depending on the output. Activation plays a big role and it is determined by the scheduler. There are generally three types of schedulers. These are (1) fully synchronous (FSYNC) scheduler where the time is divided into global rounds and each robot activates in each round and simultaneously executes their LCM cycle; (2) Semi synchronous (SSYNC) scheduler where also the time is divided into global rounds but some robots activate in a round and simultaneously execute

Table 1
Comparison Table.

Work	Visibility range (hop)	Scheduler	Door number	Graph Topology	Internal Memory	Color number
1 st algorithm in [12]	2	ASYN	1	port labeled rectangular grid	Each robot can recognize the node it came from	3
2 nd algorithm in [12]	3	ASYN	1	unoriented rectangular grid	None	7
1 st algorithm in [13]	3	ASYN	1	Arbitrary connected Graph	$O(\log(\Delta))$	$\Delta + 6$
2 nd algorithm in [13]	5	SSYN	$k > 1$	Arbitrary connected Graph	$O(\log(\Delta + 6))$	$\Delta + k + 6$
Our Algorithm	2	SSYN	None (Arbitrary initial deployment)	oriented rectangular grid	None	3

their LCM cycle; (3) Asynchronous (ASYN) scheduler where there is no common notion of time among robots and all robots execute their LCM cycle independently.

Vision is an important factor in performing these tasks. In [2–5] infinite visibility has been used. But infinite visibility is not practically possible due to hardware limitations. Limited visibility is more practical. Under limited visibility, a robot can see up to a certain distance in a plane and up to a certain hop in discrete space. In our work, we consider $\mathcal{LU}\mathcal{MI}$ model robots with two hop visibility range and three colors under a semi-synchronous scheduler. The robots agree on the two directions and their orientations, one which is parallel to rows of $\mathcal{G}$ and another which is parallel to the columns of $\mathcal{G}$. Hence each robot can determine its four directions. In this work, we propose a $\mathcal{MAXIS}$ formation algorithm for a robot swarm, which is initially placed arbitrarily on the nodes of the grid. We show that the proposed algorithm forms $\mathcal{MAXIS}$ under a semi-synchronous scheduler using robots having only two hop visibility and a light that can take three distinct colors. The next section describes all relevant works and discusses the scope of our work.

1.1. Related works and our contributions

Using swarm robotics various types of problems have been studied like exploration [5,6], gathering [4,7], dispersion [8–10], pattern formation [2,3,11] under different model. In [2–5] robots are considered to have infinite visibility. But infinite visibility is not practically possible due to hardware limitations. Limited visibility is more practical. Robots with limited visibility are called *myopic* robots. Such a myopic robot has a finite visibility range say, δ . Any two robots having visibility range δ and having distance from at most δ from each other can communicate with each other. Note that, the visibility range (or also popularly known as communication radius) is different than the sensing radius of the robots.¹ In discrete setting, where the robots are placed on a graph, the visibility radius is generally measured as hop-visibility. The robots have k -hop visibility means that two robots can communicate with each other if distance between the nodes where they are located is at most k in the graph.

Myopic robots have been used in [7,12,13]. A lot of problems [2,4,5,12,9,14,15] have been explored under the grid graph. $\mathcal{MAXIS}$ formation on a finite grid can be seen from two perspectives. One perspective is the deployment of robots through a door node. Another perspective is pattern formation. As of our knowledge, there is no algorithm for arbitrary pattern formation in a finite grid graph. To the best of our knowledge, there are only two reported works [12,13] which considers $\mathcal{MAXIS}$ formation problem on a graph using an autonomous robot swarm. The authors [12] have given a $\mathcal{MAXIS}$ filling algorithm using robots having light with three colors, two hop visibility for port labeled finite grid under asynchronous scheduler. In this algorithm, also authors considered the assumption that each robot can recognize the node it came from when at its current node. The authors suggest that these assumptions can be implemented by introducing an internal light that can take four different colors. In another algorithm, they solved the same problem using robots with seven light colors, and three hop visibility under an asynchronous scheduler but in an unoriented grid. Let Δ denote the maximum degree of a graph. The researchers [13] have given a $\mathcal{MAXIS}$ filling algorithm for an arbitrary graph with one door node using $(\Delta + 6)$ light color, three hop visibility, $O(\log(\Delta))$ bits of persistent storage under asynchronous scheduler. In another algorithm, they have solved the same problem with $k(> 1)$ door nodes using $(\Delta + k + 6)$ light color, five hop visibility, $O(\log(\Delta + k))$ bits of persistent storage under semi synchronous scheduler. Another set of works is [14,15] which are remotely related to $\mathcal{MAXIS}$ formation problem. The study [15] solves the uniform scattering problem under an asynchronous scheduler and [14] solves the uniform scattering problem under a fully synchronous scheduler on a finite rectangular grid considering myopic robots. However, $\mathcal{MAXIS}$ formation can not be achieved by any special case or slight modification of these works.

In this paper from the motivation of finding a robust but cost-effective coverage of a rectangular region, we give an algorithm to form a $\mathcal{MAXIS}$ pattern on a rectangular finite grid by luminous robots under a semi-synchronous scheduler. In contrast to [12,13], our proposed algorithm does not use the door concept and allows to form of a $\mathcal{MAXIS}$ starting from any initial configuration. Thus, this work generalizes the initial condition of the work in [12,13] for rectangular grid topology. Also, there can be practical scenarios where the door concept is not possible to implement. Suppose the robots are arbitrarily placed on the grid initially. If one wants to convert it to a door concept scenario then all robots need to gather at a corner, which might not be possible if the robots are not point robots. One might argue that all initial positions of robots can be considered as different doors and compare it with the multi-door

¹ The communication radius of wireless sensors is generally kept at least the sensing radius.

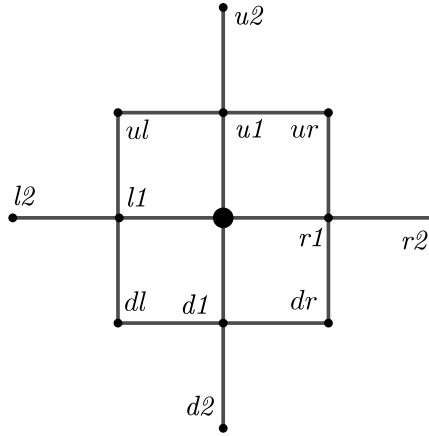


Fig. 2. View of a robot with two hop visibility.

algorithm of [13] which works under a semi-synchronous scheduler. But compared with that, our algorithm uses only a constant memory. The proposed algorithm in our work uses robots having lights that can take only three colors. The multi-door algorithm in [13] uses five hop visibility for robots whereas our algorithm only uses two hop visibility. A comparison Table 1 (In this table, Δ denotes the maximum degree of a graph) is presented to clarify the scope of this work.

Outline of the paper Section 2 discusses the model and provides the problem definition. Section 3 presents our proposed algorithm and also proves its correctness. Finally, Section 4 gives the concluding remarks and discusses the future scope of our work.

2. Model and problem definition

Consider a rectangular grid graph $\mathcal{G} = P_m \times P_n$, where P_m and P_n are path graphs with size $m > 1$ and $n > 1$ respectively. Let $\mathcal{G}$ be embedded on $\mathbb{R}^2$, such that the nodes of the graphs are placed at the coordinates $\{(x, y) | x = 1, \dots, m \text{ and } y = 1, \dots, n\}$ with respect to a coordinate system. We consider the robots equipped with motion actuators and visibility sensors. These robots move on $\mathcal{G}$. Robots can stay on the nodes only. Robots can sense their surrounding nodes and can move through edges. We call the topmost row as 1^{st} row, then the second row from the top as 2^{nd} row, and so on. Similarly, we call the leftmost column as 1^{st} column, then the second column from the left as 2^{nd} column, and so on. We can think the grid as an $m \times n$ matrix, where the $(i, j)^{th}$ entry of the matrix represents the node on i^{th} row and j^{th} column of the grid. We assume that the size of the rectangular grid is unknown to the robots. We consider the leftmost column, rightmost column, uppermost row, and lowermost row as the left boundary, right boundary, upper boundary, and lower boundary respectively.

Each robot on activation executes a look-compute-move (LCM) cycle. In the look phase, a robot takes a snapshot of its surrounding in its vicinity. In compute phase it runs an inbuilt algorithm taking the snapshot and its previous state (if the robot is not oblivious) as an input. Then it gets a color and a position as output. In the move phase, the robot changes its color if needed and moves to its destination node or stays at the same node.

Scheduler: There are generally three types of schedulers, which are fully synchronous, semi-synchronous, and asynchronous. In a synchronous scheduler, the time is equally divided into different rounds. The robots activated in a round execute the LCM cycle and each phase of the LCM cycle is executed simultaneously by all the robots. That means, all the active robots in a round take their snapshot at the same moment, and, Compute phase and Move phase of all active robots take the same time. Under a fully synchronous scheduler, each robot gets activated and executes the LCM cycle in every round. Under semi synchronous scheduler, a nonempty set of robots gets activated in a round. An adversary decides which robot gets activated in a round. In a fair adversarial scheduler, each robot gets activated infinitely often. Under an asynchronous scheduler, there is no common notion of time for the robots. Each robot independently gets activated and executes its LCM cycle. The time length of LCM cycles, Compute phases and Move phases of robots may be different. The time length of an LCM cycle of a robot is finite but can be unpredictably long. In this work, robots are working under a fair semi synchronous scheduler.

Axes Agreement: All robots agree on the directions of the axis parallel to rows and the axis parallel to columns. Hence robots agree on the global notion of up, down, right, and left directions. Each robot can determine the four directions from a node.

Visibility of robots: A robot can see (or, communicate) all of its neighbor nodes within two hop distance. Thus a robot can see 13 nodes including its position.

The first left neighbor node, second left neighbor node, first upward neighbor node, second upward neighbor node, first right neighbor node, second right neighbor node, first downward neighbor node, second downward neighbor node, upward-right neighbor node, upward-left neighbor node, downward-right neighbor node, downward-left neighbor node of a robot are denoted by $l1$, $l2$, $u1$, $u2$, $r1$, $r2$, $d1$, $d2$, ur , ul , dr , dl respectively (See Fig. 2).

Lights: Each robot has a light that can take three different colors. These colors are red, blue and green. The initial color of each robot is green. The blue color indicates that the robot wants to move but its desired path is stuck by other robots. The red

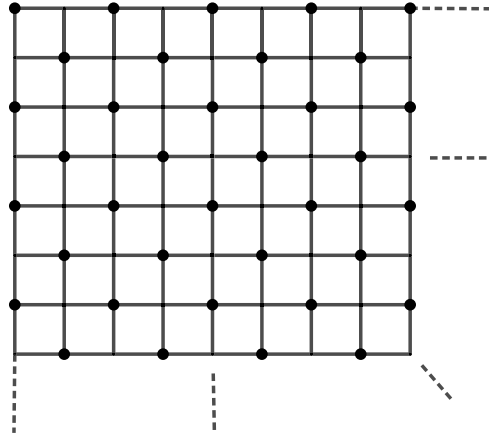


Fig. 3. Black dots represent the nodes of the set S .

color indicates that the robot has reached its final position and will not move further. Here onward we shall call a robot with color red (or blue or green) as red (or blue or green) robot.

Definition 1 (*Maximum Independent Set*). An independent set I of a graph $\mathcal{G}$ is a set of nodes of $\mathcal{G}$ such that no two nodes of that set are adjacent. A maximum independent set ($\mathcal{MA\mathcal{X}IS}$) of $\mathcal{G}$ is an independent set of the largest possible size.

We consider a set S of $(i, j)^{th}$ grid nodes having coordinates such that $i \equiv j \pmod{2}$. The nodes of the set S are depicted in Fig. 3. One can calculate that S contains $\lceil \frac{mn}{2} \rceil$ nodes. In next Proposition 1 we state that S is a $\mathcal{MA\mathcal{X}IS}$ of $\mathcal{G}$.

Proposition 1. *The set S of nodes described above forms a $\mathcal{MA\mathcal{X}IS}$ of $\mathcal{G}$.*

Proof. It is very easy to verify that no two nodes of S are adjacent, so S is an independent set of $\mathcal{G}$. If possible let there be an independent set of sizes more than $|S|$. Then there is an independent set S' of $\mathcal{G}$ of size $p_0 = |S| + 1$. Now we have two exhaustive cases; either mn is even or mn is odd.

Case-I: (mn is even) Without loss of generality we assume that n is even. In this case $p_0 = \frac{mn}{2} + 1$. From pigeon hole principle there is at least one row which consist $\lceil \frac{p_0}{m} \rceil = \frac{n}{2} + 1$ nodes of S' . If a row consists $\frac{n}{2} + 1$ nodes of S' , then there will be at least two nodes of S' on that row that are adjacent to each other. This contradicts the fact that S' is an independent set.

Case-II: (mn is odd) In this case, m and n both are odd. Therefore $p_0 = \frac{mn+1}{2} + 1$. Since S' is an independent set, so a row of $\mathcal{G}$ can contain at most $\frac{n+1}{2}$ nodes of S' . We call the rows containing $\frac{n+1}{2}$ nodes of S' as a row of *type-A*. We call the rows containing less than $\frac{n+1}{2}$ nodes of S' as a row of *type-B*. Let there be k rows of *type-A*, then there are $m - k$ rows of *type-B*. Since S' is an independent set, so all rows cannot be of *type-A*. Therefore $m - k > 0$. Now, in total *type-A* rows contain $k \times \frac{n+1}{2}$ nodes of S' . Therefore $m - k$ rows of *type-B* contain remaining

$$\frac{mn+1}{2} + 1 - (k \times \frac{n+1}{2}) = \frac{(m-k)n+3-k}{2}$$

nodes of S' . Then from pigeon hole principle there is at least a row of *type-B* that contains

$$\lceil \frac{(m-k)n+3-k}{2(m-k)} \rceil > \frac{n-1}{2}$$

nodes of S' , which is a contradiction.

Hence there is no independent set of $\mathcal{G}$ having a size more than $|S|$. Therefore S is an $\mathcal{MA\mathcal{X}IS}$ of $\mathcal{G}$. Hence in total at most $\frac{n+1}{2} \times \frac{m+1}{2}$ many nodes of S' can be there in all rows of *type-A*. Hence total at least $\frac{mn+1}{2} + 1 - \frac{n+1}{2} \times \frac{m+1}{2} = \frac{n-1}{2} \times \frac{m-1}{2} + 1$ nodes of S' can be in all row of *type-B*. $\square$

We define u_i as the number of nodes of S present in the i^{th} row of the grid. If n is even then $u_i = \frac{n}{2}$. For odd n , $u_i = \frac{n+1}{2}$ if i is odd and $u_i = \frac{n-1}{2}$ if i is even. We assume that initially $|S| = \lceil \frac{mn}{2} \rceil$ robots are present arbitrarily on different nodes of the rectangular grid such that there can be at most one robot on a node of the rectangular grid. Next, we state the problem formally.

Definition 2 (*$\mathcal{MA\mathcal{X}IS}$ formation problem*). Suppose a set of finite robots are placed arbitrarily at distinct nodes of a finite rectangular grid $\mathcal{G}$. The $\mathcal{MA\mathcal{X}IS}$ formation problem requires the robots to occupy distinct nodes of $\mathcal{G}$ and settle down avoiding collision such that the set of occupied nodes of $\mathcal{G}$ is a maximum independent set of $\mathcal{G}$.

	Green robot
	Blue robot
	Red robot
	The node may or may not exist
	The node does not exist
	The node exists

Fig. 4. Meaning of the symbols used in different diagrams.

The next section provides a proposed algorithm that solves *MAXIS* formation problem and the algorithm is collision free.

3. *MAXIS* formation algorithm

This section provides an algorithm namely, *MAXIS* Formation Algorithm that claims to solve the *MAXIS* formation problem. Different views of a robot are depicted in different figures in this section. In the figures of this section onward green, blue and red color filled circles respectively represent green robot, blue robot and red robot. The black circle indicates a node that may or may not exist. If that node exists then it can be vacant or occupied by a robot. This means a robot can ignore black circle nodes in compute phase. The black cross indicates that the node does not exist. The black diamond indicates that the node exists. A summary has been given in table form later (See Fig. 4). The empty nodes indicates the empty nodes of a grid. If any robot has to move according to its view, then the direction of movement and its color before movement have been shown in the corresponding view with a colored arrow. Initially, all robots are colored green.

A high-level idea of the algorithm: Initially green robots are arbitrarily placed on different nodes of the rectangular grid. The number of green robots is exactly same as the size of the maximum independent set of the rectangular grid. A green robot becomes red when it reaches a suitable position to form the maximum independent set. The green robots move left keeping at least two hop distance from the robots at left. Then the green robots reach left boundary or near red robots. Then the green robots move upward keeping at least two hop distance from the robots at upside. Then the green robots reach near red robots and cannot move further. Then a green robot changes its color into red. Note that, the green robot which will reach the topmost node of left boundary, will become red first. Thus the formation of the maximum independent set will start from the left most and top most corner. The formation will spread toward right and downward. A green robot becomes blue to indicate that it cannot move left or upside and it has to move right or downside but it cannot due to presence of other robots. Seeing the blue robot, other robots will try to vacant their nodes for the blue robot.

Definition 3 (Upper-Left Quadrant). Let a robot c_1 be at $(i, j)^{th}$ node of a grid. Then the nodes having coordinates $\{(x, y) : x \leq i, y \leq j\} \setminus \{(i, j)\}$ are called upper-left quadrant of c_1 .

A green robot moves at left by maintaining at least two hop distance from its left robot until it reaches the left boundary. After reaching the left boundary it moves upward by keeping at least two hop distance from its upward robot. In this way, a robot will be fixed at the upper left corner node and will be fixed first. green robots move left by maintaining the necessary distance from their left robot until it reaches the left boundary or near a red robot (See Fig. 7).

Then it moves upward by maintaining the necessary distance from its upward robot until it reaches the upper boundary or near a red robot (See Fig. 13).

Thus the robot reaches a suitable node from which it can see the necessary view to becoming red (See Fig. 18).

Definition 4 (Fixed robot). When a robot becomes red, it does not move any more according to the *MAXIS* Formation Algorithm 1. This robot is called a fixed robot.

If a green robot c_a sees that its $l1$ (if c_a is at upper boundary) or $u1$ (if c_a is at left boundary) or both (if c_a is neither at upper boundary nor at left boundary) neighbor nodes are occupied by red robots and c_a can not move upward or left then there are two possibilities.

Case-1: If $r1$ (if c_a is not at right boundary) or $d1$ (if c_a is at right boundary) neighbor node of c_a is vacant then it will move one hop right (if c_a is not at right boundary) or one hop down (if c_a is at right boundary) (See Fig. 11 or Fig. 8).

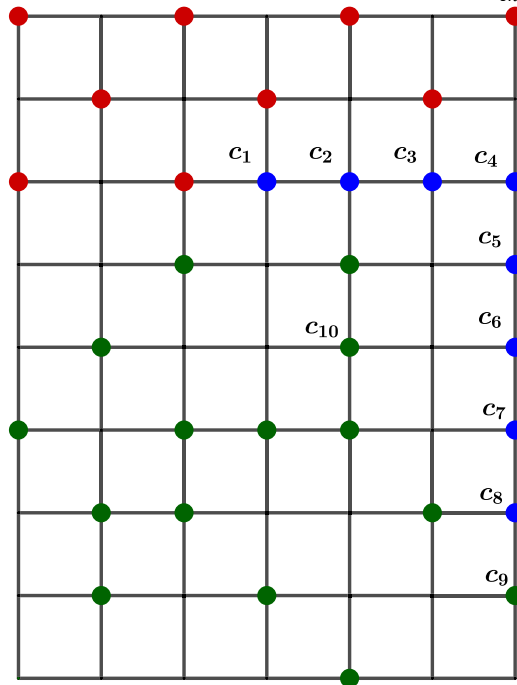


Fig. 5. Blue Sequence and Adjacent green Robot.

Case-2: If $r1$ (if c_a is not at right boundary) or $d1$ (if c_a is at right boundary) neighbor node of c_a is occupied by a robot then c_a will turn into blue indicating that it has been stuck and wants to move right (if c_a is not at right boundary) or down (if c_a is at right boundary) but can not move (See Fig. 15).

Now if a green robot c_b sees its ll neighbor node is occupied by a blue robot c_a then there are following cases.

Case-1: If $r1$ (if c_b is not at right boundary) neighbor node of c_b is vacant or $d1$ (if c_b is at right boundary) neighbor node of c_b is vacant then it will move one hop right (if c_b is not at right boundary) or one hop down (if c_b is at right boundary) (See Fig. 12 or Fig. 9).

Case-2: If $r1$ (if c_b is not at right boundary) neighbor node of c_b is not vacant or $d1$ (if c_b is at right boundary) neighbor node of c_b is not vacant.

Case-2.1: If $u1$ and $u2$ neighbor node of c_b is vacant then c_b will move upward by keeping necessary distance from its upward robot until it reaches upper boundary or near a red robot (See Fig. 14).

Case-2.2: If $u1$ or $u2$ neighbor node of c_b is occupied by a non-red robot then c_b will do nothing.

Case-2.3: If one of $u1$ and $u2$ neighbor nodes of c_b is occupied by a `red` robot and another is vacant then it will turn `blue` (See Fig. 16).

If a green robot c_d which is on the right boundary sees its $u1$ neighbor node is occupied by a blue robot c_c then there are the following cases.

Case-1: $l1$ and $l2$ neighbor node of c_d both are vacant then c_d will move one hop left.

Case-2: Anyone between $l1$ and $l2$ neighbor nodes of c_d or both are non vacant.

Case-2.1: If $d1$ neighbor node of c_d is vacant then c_d will move one hop down (See Fig. 10).

Case-2.2: If $d1$ neighbor node of c_d is occupied by a green robot then c_d will turn into blue (See Fig. 17).

The green robots which will become blue, will be on consecutive nodes. These blue robots will form a sequence like structure. We define this sequence as “Blue Sequence” in the following definition. Such definition has been introduced because the term “blue sequence” has to be repeated several times thereafter.

Definition 5 (Blue Sequence). Consider a blue robot c such that either (i) its left node (if exists) is not occupied by a blue robot if c is not at the right boundary, or (ii) both of its left node and upward node (if exists) are not occupied by a blue robot if c is at the right boundary. For case-(i) scan the grid from c to its rightwards. When it hits the right boundary then scan downwards. For case-(ii) scan the grid from c to downwards. For both cases stop scanning when it reaches a node that is not occupied by a blue robot. The trail of blue robots in the scanning is called a blue sequence.

In Fig. 5 the robots $c_1, c_2, c_3, c_4, c_5, c_6, c_7$, and c_8 form the blue sequence.

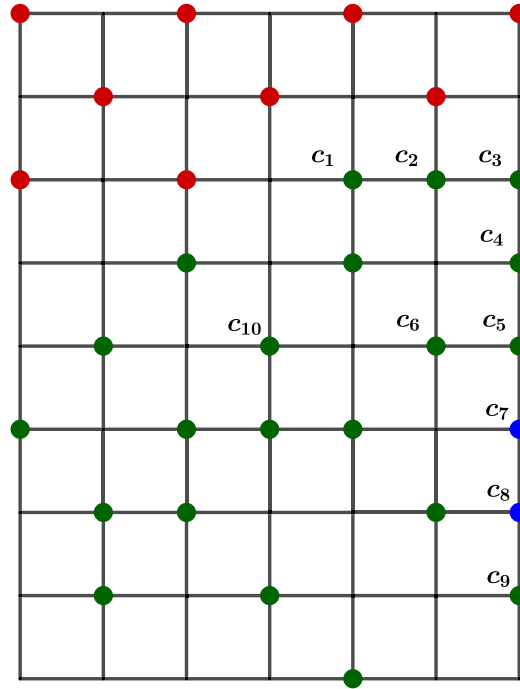


Fig. 6. 1 hop shifting has been done and Tail.

A blue sequence can proceed till the $(m-1, n)^{th}$ node at most. If the $(m, n)^{th}$ node is occupied by some robot, that robot will never be blue since its $d1$ neighbor node does not exist and the existence of $d1$ neighbor node is necessary to become blue for the robots present in the right boundary.

Definition 6 (Adjacent Green Robot). Consider the blue sequence will not proceed further. If the blue sequence ends before the right boundary then the green robot at the $r1$ neighbor node of the rightmost blue robot of the sequence and if the blue sequence continues through right boundary then the green robot at the $d1$ neighbor node of the downmost blue robot of the sequence present in the right boundary is called the adjacent green robot of the blue sequence.

In Fig. 5 c_9 is the adjacent green robot of the blue sequence.

Definition 7 (Predecessor Blue Robots). Consider a blue robot c_k of a blue sequence. All the robots which became blue in that blue sequence before the round in which c_k became blue, are called the predecessor blue robots of c_k in that blue sequence.

In Fig. 5 c_1, c_2, c_3, c_4 , and c_5 are the predecessor blue robots of c_6 .

If a blue robot c_e which is not at the right boundary sees its $r1$ neighbor node is vacant then it turns green and moves one hop right (See Fig. 19).

If a blue robot c_e which is at the right boundary sees its $l1$, $l2$, and $d1$ neighbor nodes then there are the following cases.

Case-1: Both $l1$ and $l2$ neighbor nodes of c_e are vacant then c_e turns green and move one hop left (See Fig. 20).

Case-2: Any one between $l1$ and $l2$ neighbor nodes of c_e is not vacant and its $d1$ neighbor node is vacant then it turns green and moves one hop down (See Fig. 21).

Definition 8 (Tail). If the blue sequence continues through right boundary and a blue robot of the sequence from the right boundary leaves the sequence by moving left then the remaining blue robots of the sequence below the leaving blue robot will be called tail.

In Fig. 6 c_7, c_8 is the tail after c_6 leaves the blue sequence.

Definition 9 (1 Hop Shifting). If the adjacent green robot or any robot of the blue sequence moves from its position then each of its predecessor blue robots moves one hop to fill the vacant node and to make the starting node of the blue sequence vacant. This is called 1 hop shifting of the blue sequence.

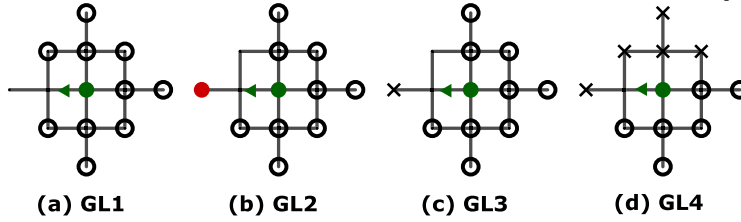


Fig. 7. Views of Green robot to move Left.

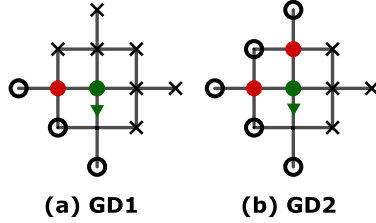


Fig. 8. Views of Green robot to move Downward due to red robot.

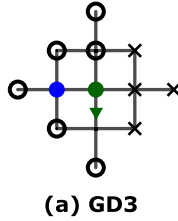


Fig. 9. View of Green robot to move Downward when a blue robot is at left.

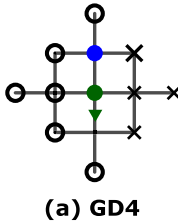


Fig. 10. View of Green robot to move Downward when a blue robot is at upward.

In Fig. 6, 1 hop shifting of the blue sequence in Fig. 5 has been done after the robot c_6 moves one hop left and makes its position vacant. The shifting is happened in the following way. First c_5 changes its color to green and moves downwards and c_4 also does the same. Then c_3 changes its color to green and moves right and, then firstly c_2 and then c_1 do the same. Hence, if no other robot moves then after a finite round the configuration turns from Fig. 5 to the Fig. 6.

If a blue robot c_e which is at the right boundary sees its $u1$ neighbor node is vacant and $l1$ neighbor node is not occupied by a blue robot then it turns green (See Fig. 22).

If a blue robot c_e which is at the right boundary sees its $u1$ neighbor node is occupied by a red robot and $l1$ neighbor node is vacant then it turns green (See Fig. 22).

If a blue robot c_e which is at the right boundary sees its $u1$ neighbor node is occupied by a green robot then it turns green (See Fig. 22).

Now we define some sets of views and then we provide the pseudo code of this algorithm in Algorithm 1.

$$G_1 = \{GL1, GL2, GL3, GL4\}$$

$$G_2 = \{GD1, GD2, GD3, GD4\}$$

$$G_3 = \{GR1, GR2, GR3, GR4, GR5, GR6, GR7, GR8\}$$

$$G_4 = \{GU1, GU2, GU3, GU4, GU5, GU6, GU7, GU8, GU9, GU10, GU11, GU12, GU13, GU14, GU15, GU16, GU17, GU18, GU19, GU20\}$$

$$G_5 = \{GB1, GB2, GB3, GB4, GB5, GB6, GB7, GB8, GB9, GB10, GB11, GB12, GB13, GB14\}$$

$$G_6 = \{G-R1, G-R2, G-R3, G-R4, G-R5, G-R6, G-R7\}$$

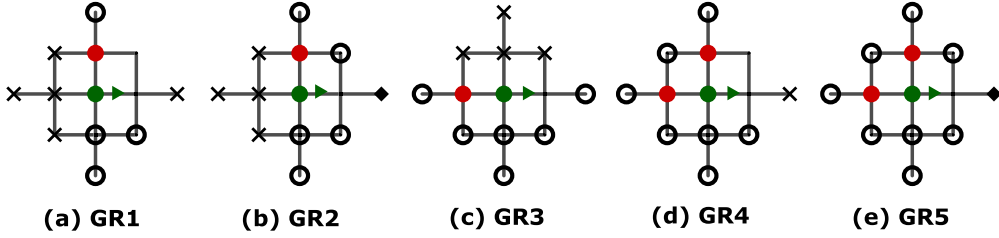


Fig. 11. Views of Green robot to move Right due to red robot.

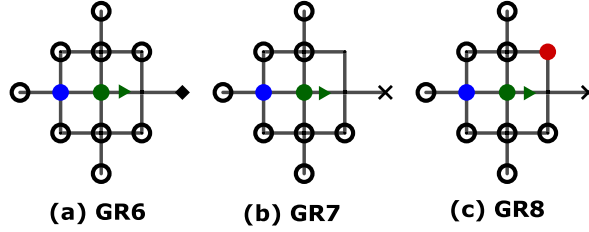


Fig. 12. Views of Green robot to move Right due to blue robot.

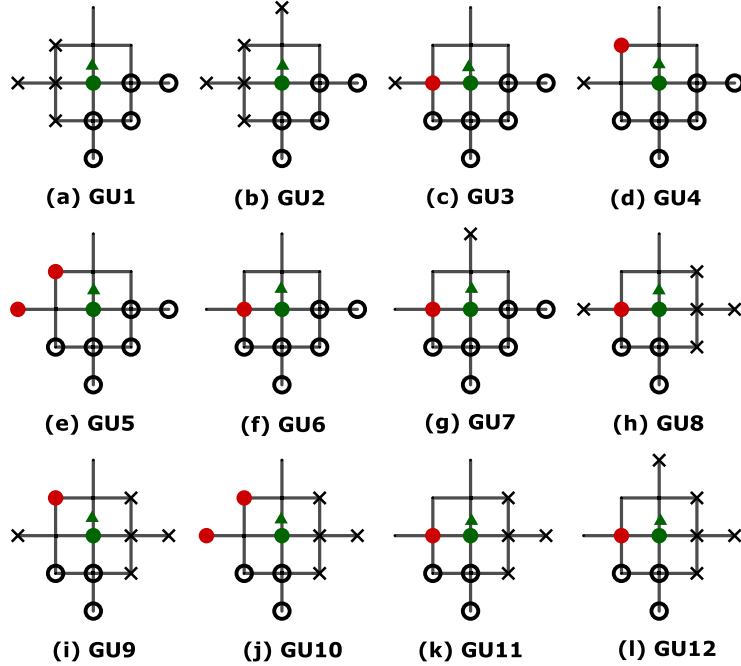


Fig. 13. Views of Green robot to move Upward due to red robot.

$$B_1 = \{\text{BGR1}, \text{BGR2}, \text{BGR3}\}$$

$$B_2 = \{\text{BGL1}\},$$

$$B_4 = \{\text{BG1}, \text{BG2}, \text{BG3}, \text{BG4}, \text{BG5}\}$$

$$B_3 = \{\text{BGD1}\}$$

3.1. Correctness proofs

In Theorem 1 we have shown that the $\mathcal{MAXIS}$ Formation Algorithm is collision free. In Lemma 1 we have shown that when all robots have turned their color to red then the set of robot occupied grid nodes forms a $\mathcal{MAXIS}$ of the grid $\mathcal{G}$. Finally in Theorem 2 we have proved that this formation will occur after finite rounds.

Theorem 1. *There are no collisions of robots while executing the $\mathcal{MAXIS}$ Formation Algorithm.*

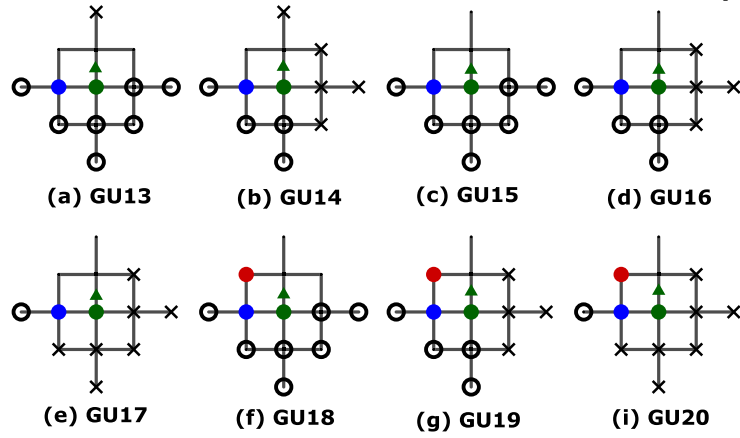


Fig. 14. Views of Green robot to move Upward due to blue robot.

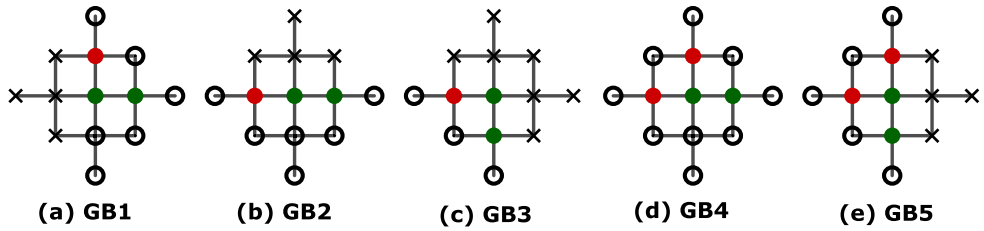


Fig. 15. Views of Green robot to become Blue due to red robot.

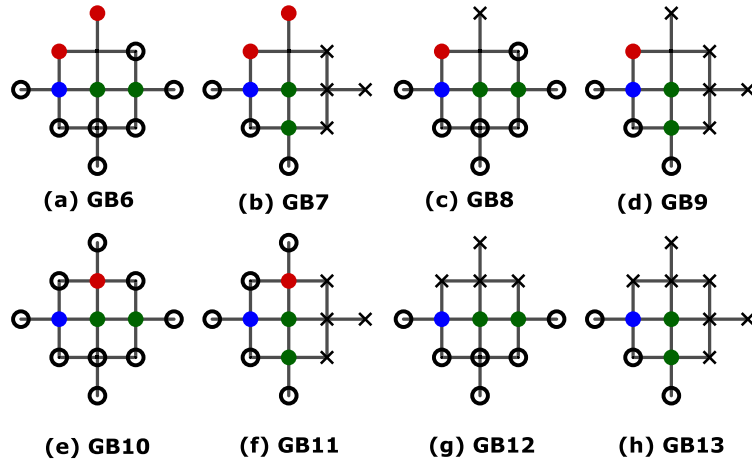


Fig. 16. Views of Green robot to become Blue when a blue robot is at left.

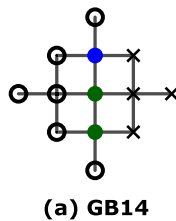


Fig. 17. View of Green robot to become Blue when a blue robot is at upward.

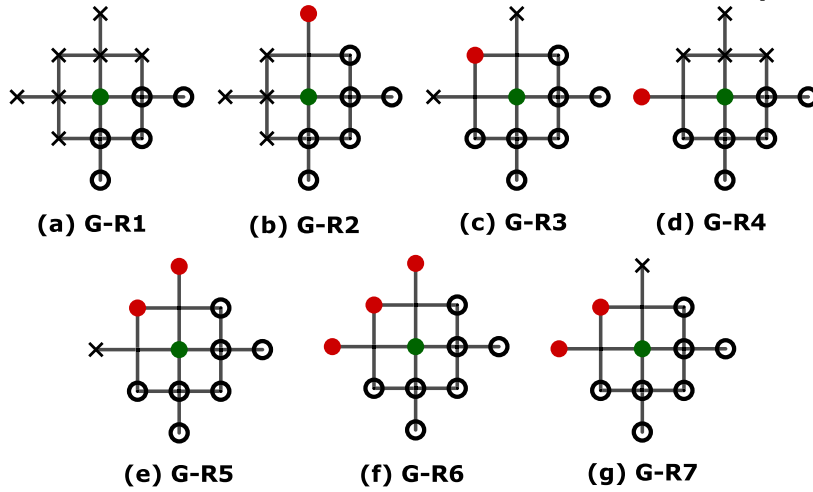


Fig. 18. Views of Green robot to become Red.

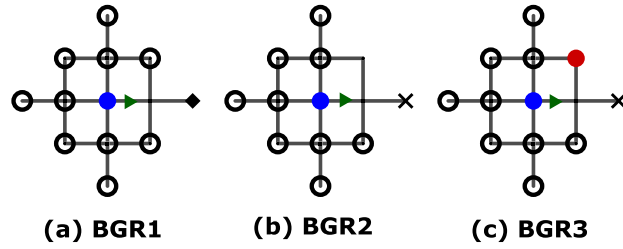


Fig. 19. Views of Blue robot to become Green and move Right.

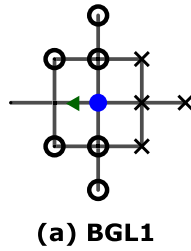


Fig. 20. View of Blue robot to become Green and move Left.

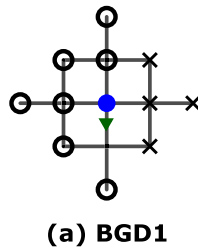


Fig. 21. View of Blue robot to become Green and move Downward.

Proof. There can be two types of collisions in a round.

Type-1: There is a robot present already in a node and another robot comes to that node.

Type-2: More than one robot, each from a different node comes to a particular vacant node.

According to the $\mathcal{MATS}$ Formation Algorithm no robot moves to a node that is already occupied. So there is no collision of Type-1.

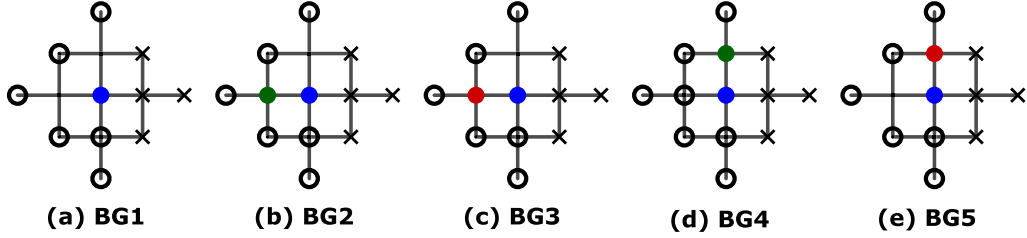


Fig. 22. Views of Blue robot to become Green.

Algorithm 1: *MAXIS* Formation.**Data:** Positions and colors of robots within two hop distance**Result:** One color and one destination point

```

if  $col(c)$  is green then
  if  $view(c) \in G_1$  then
    | Move one hop left
  else if  $view(c) \in G_2$  then
    | Move one hop downward
  else if  $view(c) \in G_3$  then
    | Move one hop right
  else if  $view(c) \in G_4$  then
    | Move one hop upward
  else if  $view(c) \in G_5$  then
    | Change color to blue
  else if  $view(c) \in G_6$  then
    | Change color to red
else if  $col(c)$  is blue then
  if  $view(c) \in B_1$  then
    | Change color to green and move one hop right
  else if  $view(c) \in B_2$  then
    | Change color to green and move one hop left
  else if  $view(c) \in B_3$  then
    | Change color to green and move one hop downward
  else if  $view(c) \in B_4$  then
    | Change color to green

```

According to the *MAXIS* Formation Algorithm, there are four types of movement of a robot i.e. left move($\mathcal{L}$), right move($\mathcal{R}$), upward move($\mathcal{U}$), and downward move($\mathcal{D}$). Since the scheduler is SSYNC, Type-2 collision can occur between two robots which are exactly two hop away from each other in the beginning of that round. Considering all possible combinations there can be six different collisions i.e. ($\mathcal{LR}$), ($\mathcal{LU}$), ($\mathcal{LD}$), ($\mathcal{RU}$), ($\mathcal{RD}$), ($\mathcal{UD}$).

($\mathcal{LR}$): A robot will move left if its view belongs to G_1 or B_2 . From Fig. 7 and Fig. 20 it is clear that for ($\mathcal{L}$) movement of a robot c , $l1$ neighbor node of c will always remain vacant and $l2$ neighbor node of c is vacant or occupied by a red robot or does not exist. There is no robot that will move to $l1$ neighbor node of c by ($\mathcal{R}$) movement. So there is no ($\mathcal{LR}$) collision.

($\mathcal{LU}$): A robot will move upward if its view belongs to G_4 . From Fig. 14 and Fig. 13 it is clear that for ($\mathcal{U}$) movement of a robot c , $u1$ neighbor node of c is vacant or does not exist. There is no robot that will move to $u1$ neighbor node of c by ($\mathcal{L}$) movement. So there is no ($\mathcal{LU}$) collision.

($\mathcal{LD}$): A robot will move downward if its view belongs to G_2 or B_3 . From Fig. 21, Fig. 9, Fig. 10 and Fig. 8 it is clear that for ($\mathcal{D}$) movement of a robot c , $d1$ neighbor node of c does not exist. There is no robot which will move to $d1$ neighbor node of c by ($\mathcal{L}$) movement. So there is no ($\mathcal{LD}$) collision.

($\mathcal{RU}$): A robot will move upward if its view belongs to G_4 . From Fig. 13 and Fig. 14 it is clear that for ($\mathcal{U}$) movement of a robot c , $u1$ neighbor node of c is vacant or occupied by a red robot or does not exist. There is no robot which will move to $u1$ neighbor node of c by ($\mathcal{R}$) movement. So there is no ($\mathcal{RU}$) collision.

($\mathcal{RD}$): A robot will move downward if its view belongs to G_2 or B_3 . From Fig. 21, Fig. 9, Fig. 10 and Fig. 8 it is clear that ($\mathcal{D}$) movement of a robot is possible through right boundary only. A robot will move right if its view belongs to G_3 or B_1 . From Fig. 12, Fig. 11, and Fig. 19 it is clear that for ($\mathcal{R}$) movement of a robot c , if $r1$ neighbor node of c is on right boundary then ur neighbor node of c is vacant or occupied by a red robot else $r1$ neighbor node of c is not on right boundary. There is no robot that will move to $r1$ neighbor node of c by ($\mathcal{D}$) movement. So there is no ($\mathcal{RD}$) collision.

($\mathcal{UD}$): A robot will move upward if its view belongs to G_4 . From Fig. 14 and Fig. 13 it is clear that for ($\mathcal{U}$) movement of a robot c , $u1$ neighbor node of c is vacant and $u2$ neighbor node of c is vacant or does not exist. There is no robot which will move to $u1$ neighbor node of c by ($\mathcal{D}$) movement. So there is no ($\mathcal{UD}$) collision. $\square$

Lemma 1. If all robots have turned their color to red then the set of robot occupied grid nodes forms a *MAXIS* of $\mathcal{G}$.

Proof. First we show that the set of robot occupied grid nodes is an independent set of G . We show this by showing that no two red robots are adjacent. Opposite to our claim, let there be two adjacent red robots c_1 and c_2 . If c_1 and c_2 are on the same column then let c_2 be the robot below c_1 and if c_1 and c_2 are on the same row then let c_2 be the robot right to c_1 . Let c_1 and c_2 change their color to red in t_1^{th} and t_2^{th} round respectively. Now there can be two possibilities.

Case-I: ($t_1 \leq t_2$) Since red robots never move, so throughout t_2^{th} round the c_1 robot is at the $u1$ or $l1$ neighbor node of c_2 . According to our proposed algorithm, c_2 will change its color to red if it sees any view belonging to the set G_6 . But no view in G_6 allows the $u1$ or $l1$ neighbor node of c_2 to be occupied by a robot. So this leads to a contradiction.

Case-II: ($t_1 > t_2$) In this case c_2 becomes red and gets fixed before c_1 . Hence the $l2$, $u2$ and $u1$ neighbors of c_2 must be occupied by red robots if these neighbor nodes exist and it sustains in t_1 round also (since red robots never move). If c_1 robot is at $u1$ (or, $l1$) neighbor node of c_2 , then $u1$ (or, $l1$) neighbor node of c_2 exists. Hence view of c_2 at t_2^{th} round must be one of **G-R2** (replace **G-R2** by **G-R4** for the case when c_1 is at $l1$ neighbor node of c_2), **G-R3**, **G-R5**, **G-R6** and **G-R7**. In all such views either $l1$ or $u1$ neighbor node of c_1 is occupied by a red robot. Hence c_1 would not change its color to red in t_1^{th} round, which is a contradiction.

Hence if all robots turn red then the robot occupied nodes form an independent set. Now the number of robots is $\lceil \frac{mn}{2} \rceil$ which is the maximum possible size of an independent set of G . Since Theorem 1 gives that there is no collision of robots, so all the red robots must be at distinct grid nodes. So the number of robot occupied nodes after all robots turned red is also $\lceil \frac{mn}{2} \rceil$. Thus, the independent set formed by robot occupied grid nodes is a **MAXIS**. $\square$

Lemma 2. If a row consists of three types of robots i.e. red, blue, and green then the red robots will be at the left, the blue robots will be in the middle and the green robots will be at the right of the row.

Proof. A green robot becomes red, when it sees its $l2$, $u1$, $u2$ neighbor nodes (if exists) are occupied by red robots and $l1$, $u1$ neighbor nodes (if exists) are vacant (See Fig. 18). So there cannot be any green or blue robot at left of a red robot. Thus red robots are at left of a row.

When a blue sequence starts then the $l1$ neighbor node (if exists) of the blue robot which became blue first, is occupied by a red robot (See Fig. 15). A blue sequence is a sequence of blue robots which are at consecutive nodes. So there is no green robot at the middle of a blue sequence.

Thus the red robots will be at left, blue robots will be at middle and green robots will be at right of the row. $\square$

Lemma 3. If there are $\lceil \frac{n}{2} \rceil - 2$ robots present in a row consists of n nodes, then after finite round n^{th} , $(n-1)^{th}$ and $(n-2)^{th}$ nodes will be vacant.

Proof. In a row the distance of a red robot from its immediate left or immediate right red robot is exactly two hop. In a row distance of a red robot from its immediate right blue robot is exactly one hop. In a row after finite round the distance of a green robot from its immediate left or immediate right green robot will be at most two hop since all green robots move left by keeping two hop distance. In a row distance of a blue robot which became blue last, from its immediate right green robot is exactly one hop. In a row distance of a blue robot from its immediate left or immediate right blue robot is exactly one hop. Thus by Lemma 2 in a row distance of a robot from its immediate left or immediate right robot is at most two hop. By the assumption, maximum possible number of robots is $\frac{n}{2} - 2$ (if n is even) or $\frac{n+1}{2} - 2$ (if n is odd). If possible we try to put the robots in such a way so that n^{th} , $(n-1)^{th}$ and $(n-2)^{th}$ node does not remain empty. If we put robots on even positioned nodes then i^{th} robot will be at $2i^{th}$ node. $(\frac{n}{2} - 2)^{th}$ robot will be at $(n-4)^{th}$ node. $(\frac{n+1}{2} - 2)^{th}$ robot will be at $(n-3)^{th}$ node. Thus in both cases n^{th} , $(n-1)^{th}$ and $(n-2)^{th}$ nodes will be vacant. $\square$

Lemma 4. Let c_1 be the leftmost non-red robot in the topmost non-red robot occupied row. Let $r1$ be a part of a blue sequence and c_1 be the first robot that turned blue for any one view from Fig. 15. If the blue sequence ends at $(m-1, n)$ node then 1 hop shifting will be done.

Proof. If the blue sequence starts at f^{th} row, continues through right boundary and ends at $(m-1, n)$ node then 1^{st} , 2^{nd} , ..., i^{th} , ..., $(f-1)^{th}$ row each contains u_i robots. If n is even then $u_i = \frac{n}{2}$. For odd n , $u_i = \frac{n+1}{2}$ if i is odd and $u_i = \frac{n-1}{2}$ if i is even. f^{th} row contains more than u_f robots. $(f+1)^{th}$, $(f+2)^{th}$, ..., m^{th} row together will contain less than $u_{f+1} + u_{f+2} + \dots + u_m$ robots. There will be at least one row (say h^{th} row) which will contain less than u_h robots. If more than one such row exists then consider the topmost row (say l^{th} row) which contains less than u_l robots. If any robot comes from the below row and makes u_l number robot in l^{th} row, then we shall consider the below q^{th} row which contains less than u_q robot. If this continues since the number of rows is constant we must get such a row (say w^{th} row) where the number of robots will be less than u_w and no robots will enter from below. Without loss of generality, we consider such a row as z^{th} row. If n is odd then u_z is $\lceil \frac{n}{2} \rceil$ when z is odd and $\lceil \frac{n}{2} \rceil - 1$ when z is even. If n is even then u_z is $\lceil \frac{n}{2} \rceil$. If we consider z^{th} row except the right boundary node which is occupied by a blue robot or green robot then there are $(n-1)$ nodes and atmost $\lceil \frac{n}{2} \rceil - 2$ robots. After a finite round when all the robots of z^{th} row except the right most blue or green robot will be at atmost two hop distance from each other, $(z, n-2)$ and $(z, n-1)$ node will be vacant by Lemma 3. Then the right most robot of z^{th} row will move from (z, n) node to $(z, n-1)$ node and 1 hop shifting will be done automatically. $\square$

Theorem 2. *MAXIS Formation Algorithm forms maximum independent set after finite rounds without any collisions.*

Proof. Consider the uppermost row which contains at least one green or blue robot. If there is no such row then every robot present in the grid is red. Therefore by Lemma 1 the proof is done.

Let there exists a row (say w^{th} row) that contains at least one green or blue robot. Let c_1 be the leftmost non-red robot on that row. c_1 can be green or blue. Note that all the robots present in the upper-left quadrant of c_1 are red and they are fixed.

Case-1: c_1 is green.

c_1 continues moving left as long as it sees any views from $\{GL1, GL2, GL3, GL4\}$ (Fig. 7). While c_1 is progressing left through the row if any green robot from below row moves upwards and comes to the left of c_1 then we will consider the new robot as c_1 . If c_1 does not see any view from $\{GL1, GL2, GL3, GL4\}$ (Fig. 7) then it must see any view from $\{GB1, GB2, GB3, GB4, GB5, GD1, GD2, GR1, GR2, GR3, GR4, GR5, GU1, GU2, GU3, GU4, GU5, GU6, GU7, GU8, GU9, GU10, GU11, GU12, G-R1, G-R2, G-R3, G-R4, G-R5, G-R6, G-R7\}$ (Fig. 15, Fig. 8, Fig. 11, Fig. 13, Fig. 18). If c_1 sees any one view from $\{GB1, GB2, GB3, GB4, GB5\}$ (Fig. 15) then it turns blue and goes to Case-2. If c_1 sees any one view from $\{GD1, GD2\}$ (Fig. 8) then it will go to its $d1$ neighbor node. Now we may get a new c_1 since there may exist some non-red robot at the left in the current row. Now, c_1 will not move to its $d1$ neighbor node and will remain c_1 since it will not get any view from $\{GD1, GD2\}$ (Fig. 8). If c_1 sees any one view from $\{GR1, GR2, GR3, GR4, GR5\}$ (Fig. 11) then it will go to its $r1$ neighbor node. Now it will not see any view from $\{GR1, GR2, GR3, GR4, GR5\}$ (Fig. 11) and $\{GD1, GD2\}$ (Fig. 8). If c_1 sees anyone view from $\{G-R1, G-R2, G-R3, G-R4, G-R5, G-R6, G-R7\}$ (Fig. 18) then it turns red. Else c_1 will see anyone view from $\{GU1, GU2, GU3, GU4, GU5, GU6, GU7, GU8, GU9, GU10, GU11, GU12\}$ (Fig. 13) and continues moving upward until it sees anyone view from $\{G-R1, G-R2, G-R3, G-R4, G-R5, G-R6, G-R7\}$ (Fig. 18). Finally, c_1 will see anyone view from $\{G-R1, G-R2, G-R3, G-R4, G-R5, G-R6, G-R7\}$ (Fig. 18) and will turn red.

Case-2: c_1 is blue.

A blue robot became blue as a part of a blue sequence. Now it is either a part of a blue sequence or a part of a tail. As c_1 is the leftmost non-red robot in the topmost non-red robot occupied row, there can be two cases.

Case-2.1: If c_1 is a part of a tail then c_1 will be at right boundary and the topmost blue robot of the tail. If we consider $l1$ and $u1$ neighbor nodes of c_1 then there can be four types of figures.

In these three types i.e. $\{BG1, BG3, BG5\}$ (Fig. 22), at least one among $l1$ and $u1$ neighbor nodes of c_1 is not occupied by a red robot and c_1 will turn into green. The robot c_1 goes to Case-1 and this c_1 will never become blue as it was the upmost robot of a tail and all the robots which are at upper-left quadrant of c_1 are red and at least one among $l1$ and $u1$ neighbor nodes of c_1 is not occupied by a red robot.

If both $l1$ and $u1$ neighbor nodes of c_1 are occupied by red robots then c_1 goes to Case 2.2 (similar to $\{GB5\}$ (Fig. 15)).

Case-2.2: If c_1 is a part of a blue sequence then c_1 is the first robot that turned blue for any one view from Fig. 15. The blue sequence can continue along the row and right boundary.

Case-2.2.1: If the blue sequence ends before $(m-1, n)$ node 1 hop shifting will be done after the adjacent green robot moves from its node. 1 hop shifting would be done before it if any blue robot of the blue sequence from the right boundary moves left.

Case-2.2.2: If the blue sequence ends at $(m-1, n)$ node 1 hop shifting will be done by Lemma 4.

Now, c_1 will turn into green and goes to Case-1. This c_1 will never become blue as its $l1$ or $u1$ neighbor node is vacant and all the robots which are at upper-left quadrant of c_1 are red.

Selecting a non-red robot we are making a non-red robot into a red robot. Since the total number of robots is finite, after a finite round all the robots will be red. Therefore by Lemma 1, the proof follows. $\square$

4. Conclusion

This work presents an algorithm that forms a Maximum Independent Set (MAXIS) on a finite rectangular grid $\mathcal{G}$ by myopic robots. If the size of a maximum independent set of $\mathcal{G}$ is p' then initially p' robots are placed arbitrarily on distinct nodes of $\mathcal{G}$. However, if there are robots of number lesser than p' then our algorithm just ends up forming an independent set. The robots are considered to be luminous and have a light that can take three distinct colors. We assume the robots agree on the global notion of north, south, east, and west direction. The robots have two hop visibility. The robots are controlled under an adversarial semi-synchronous scheduler. In contrast to the previous MAXIS formation algorithms, the algorithm proposed in this work does not use the door concept. It allows the robots to form MAXIS from any arbitrary starting configuration. This generalizes the initial condition of the previous works for rectangular grid topology.

In this work, we assumed two hop visibility of robots, so as a future direction one can try proposing a MAXIS formation algorithm which only uses one hop visibility of robots. Further, it will be interesting to provide an algorithm for the same problem under an asynchronous scheduler.

CRedit authorship contribution statement

Raja Das: Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Conceptualization. **Avisek Sharma:** Writing – review & editing, Validation, Supervision, Investigation. **Buddhadeb Sau:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The first author is supported by Council of Scientific & Industrial Research (CSIR), Govt. of India, grant no. 09/096(1031)/2020-EMR-I. The second author is supported by University Grants Commission (UGC), Govt. of India, grant no. 201610019984. The third author is supported by Science and Engineering Research Board (SERB), Govt. of India, grant no. MTR/2021/000835.

Data availability

No data was used for the research described in the article.

References

- [1] R. Das, A. Sharma, B. Sau, Maximum independent set formation on a finite grid by myopic robots, in: K. Arai (Ed.), *Intelligent Computing*, Springer Nature, Switzerland, Cham, 2023, pp. 1247–1267.
- [2] K. Bose, R. Adhikary, M.K. Kundu, B. Sau, Arbitrary pattern formation on infinite grid by asynchronous oblivious robots, *Theor. Comput. Sci.* 815 (2020) 213–227, <https://doi.org/10.1016/j.tcs.2020.02.016>.
- [3] K. Bose, M.K. Kundu, R. Adhikary, B. Sau, Arbitrary pattern formation by asynchronous opaque robots with lights, *Theor. Comput. Sci.* 849 (2021) 138–158, <https://doi.org/10.1016/j.tcs.2020.10.015>.
- [4] G. D'Angelo, G.D. Stefano, R. Klasing, A. Navarra, Gathering of robots on anonymous grids and trees without multiplicity detection, *Theor. Comput. Sci.* 610 (2016) 158–168, <https://doi.org/10.1016/j.tcs.2014.06.045>.
- [5] S. Devismes, A. Lamani, F. Petit, P. Raymond, S. Tixeuil, Terminating exploration of A grid by an optimal number of asynchronous oblivious robots, *Comput. J.* 64 (1) (2021) 132–154, <https://doi.org/10.1093/comjnl/bxz166>.
- [6] S. Nagahama, F. Ooshita, M. Inoue, Ring exploration of myopic luminous robots with visibility more than one, in: M. Ghaffari, M. Nesterenko, S. Tixeuil, S. Tucci, Y. Yamauchi (Eds.), *Stabilization, Safety, and Security of Distributed Systems - 21st International Symposium, Proceedings, SSS 2019, Pisa, Italy, October 22–25, 2019*, in: *Lecture Notes in Computer Science*, vol. 11914, Springer, 2019, pp. 256–271.
- [7] S. Kamei, A. Lamani, F. Ooshita, S. Tixeuil, K. Wada, Gathering on rings for myopic asynchronous robots with lights 153 (2019) 27, <https://doi.org/10.4230/LIPIcs.OPODIS.2019.27>.
- [8] A.D. Kshemkalyani, A.R. Molla, G. Sharma, Fast dispersion of mobile robots on arbitrary graphs, in: F. Dressler, C. Scheideler (Eds.), *Algorithms for Sensor Systems - 15th International Symposium on Algorithms and Experiments for Wireless Sensor Networks, ALGOSENSORS 2019, Munich, Germany, September 12–13, 2019*, in: *Lecture Notes in Computer Science*, vol. 11931, Springer, 2019, pp. 23–40, Revised Selected Papers.
- [9] A.D. Kshemkalyani, A.R. Molla, G. Sharma, Dispersion of mobile robots on grids, in: M.S. Rahman, K. Sadakane, W. Sung (Eds.), *WALCOM: Algorithms and Computation - 14th International Conference, Proceedings, WALCOM 2020, Singapore, March 31 - April 2, 2020*, in: *Lecture Notes in Computer Science*, vol. 12049, Springer, 2020, pp. 183–197.
- [10] A.D. Kshemkalyani, A.R. Molla, G. Sharma, Efficient dispersion of mobile robots on dynamic graphs, in: *40th IEEE International Conference on Distributed Computing Systems, ICDCS 2020, Singapore, November 29 - December 1, 2020*, IEEE, 2020, pp. 732–742.
- [11] P. Flocchini, G. Prencipe, N. Santoro, P. Widmayer, Arbitrary pattern formation by asynchronous, anonymous, oblivious robots, *Theor. Comput. Sci.* 407 (1–3) (2008) 412–447, <https://doi.org/10.1016/j.tcs.2008.07.026>.
- [12] S. Kamei, S. Tixeuil, An asynchronous maximum independent set algorithm by myopic luminous robots on grids, *Comput. J.* 67 (1) (2024) 57–77, <https://doi.org/10.1093/COMJNL/BXAC158>.
- [13] S. Pramanick, S.V. Samala, D. Pattanayak, P.S. Mandal, Filling MIS vertices of a graph by myopic luminous robots 13776 (2023) 3–19, https://doi.org/10.1007/978-3-031-24848-1_1.
- [14] P. Poudel, G. Sharma, Time-optimal uniform scattering in a grid, in: R.C. Hansdah, D. Krishnaswamy, N.H. Vaidya (Eds.), *Proceedings of the 20th International Conference on Distributed Computing and Networking, ICDCN 2019, Bangalore, India, January 04–07, 2019*, ACM, 2019, pp. 228–237.
- [15] P. Poudel, G. Sharma, Fast uniform scattering on a grid for asynchronous oblivious robots, in: S. Devismes, N. Mittal (Eds.), *Stabilization, Safety, and Security of Distributed Systems - 22nd International Symposium, Proceedings, SSS 2020, Austin, TX, USA, November 18–21, 2020*, in: *Lecture Notes in Computer Science*, vol. 12514, Springer, 2020, pp. 211–228.